

SOFTVERSKI AGENTI

Predlog projekta 2026

# Specifikacija projekta

Aktorski radni okvir

**TIM**

Luka Milovanov, RA105/2022

Aleksa Bijelić, RA35/2022

Ciljna ocjena: **8**

*Aktorski radni okvir – osnovne funkcionalnosti*

## Zadatak

---

Cilj projekta je implementacija generičkog aktorskog radnog okvira (Actor Framework-a) i njegove demonstracije kroz distribuirani sistem za obradu taskova.

Razvijeni framework nije specifičan za domen problema, već omogućava kreiranje i izvršavanje različitih aktorskih sistema zasnovanih na asinhronoj komunikaciji.

Demonstracija sistema biće realizovana kroz **distributed task processing system**, u kome se poslovi (taskovi) distribuiraju između više aktora koji rade na različitim mašinama/procesima.

## Implementacija aktorskog radnog okvira

---

### Opis sistema

Framework implementira osnovne koncepte aktorskog modela:

- aktore kao nezavisne jedinice izvršavanja
- asinhronu komunikaciju putem poruka
- mailbox
- promenu ponašanja (state/behavior switching)
- lifecycle događaje (kreiranje i gašenje)
- udaljenu komunikaciju između aktora na različitim mašinama

### Komponente sistema

#### ActorSystem

Centralna komponenta koja upravlja:

- kreiranjem i uništavanjem aktora
- registracijom aktora
- rutiranjem poruka
- upravljanjem lokalnim i udaljenim aktorima

#### Actor

Apstraktna jedinica izvršavanja koja:

- poseduje jedinstveni identifikator
- ima sopstveni mailbox
- obrađuje poruke asinhrono
- može menjati ponašanje (behavior switching)
- reaguje na lifecycle događaje

#### Mailbox

Thread-safe struktura (BlockingQueue) koja čuva dolazne poruke aktora do njihove obrade.

## Message

Osnovna jedinica komunikacije između aktora. Svaka poruka sadrži:

| Polje      | Tip    | Opis                                    |
|------------|--------|-----------------------------------------|
| senderId   | PID    | Identifikator pošiljaoca                |
| receiverId | PID    | Identifikator primaoca                  |
| type       | String | Tip poruke (npr. TASK, RESULT, STATUS)  |
| payload    | Object | Sadržaj poruke (serijalizovani objekat) |

## Remote Communication Module

Omogućava komunikaciju između aktora na različitim mašinama/procesima koristeći TCP socket komunikaciju i serijalizaciju poruka.

## Implementirane funkcionalnosti

### Obavezne funkcionalnosti

| Funkcionalnost              | Opis implementacije                                                                         |
|-----------------------------|---------------------------------------------------------------------------------------------|
| Aktori                      | Implementacija i upravljanje aktorima unutar ActorSystem-a.                                 |
| Asinhrono slanje i primanje | Aktori komuniciraju bez blokiranja koristeći mailbox sistem.                                |
| Sanduče (Mailbox)           | Svaki aktor poseduje thread-safe queue za poruke.                                           |
| Menjanje ponašanja          | Aktori mogu menjati svoje ponašanje u zavisnosti od trenutnog stanja (behavior switching).  |
| Lifecycle događaji          | Kreiranje aktora, pokretanje aktora, gašenje aktora, reakcije na lifecycle evente           |
| Udaljena komunikacija       | Aktori mogu slati i primiti poruke između različitih mašina/procesa putem TCP komunikacije. |

## Demonstracija – Distributed Task Processing System

### Opis demonstracije

Za demonstraciju svih elemenata okvira implementira se distribuirani sistem za obradu taskova u kome master aktor distribuira poslove između više worker aktora koji mogu biti pokrenuti na različitim mašinama.

## Tipovi aktora

| Aktor                 | Uloga                                                                                                      |
|-----------------------|------------------------------------------------------------------------------------------------------------|
| MasterActor           | Prima taskove od klijenta, distribuira ih workerima, prati status izvršavanja, sakuplja rezultate          |
| WorkerActor           | Prima taskove od Master-a, izvršava zadate operacije, vraća rezultate Master-u, menja stanje (IDLE / BUSY) |
| ResultCollectorActor  | Skuplja i loguje rezultate, prikazuje finalni output po završetku svih taskova                             |
| LoggerActor (opciono) | Loguje lifecycle događaje i sistemske poruke                                                               |

## Taskovi

Taskovi predstavljaju poruke koje opisuju posao koji treba izvršiti.

Primeri taskova:

- izračunavanje sume brojeva
- računanje faktoriijela
- obrada stringa (reverse/uppercase)
- simulacija rada (sleep task)

## Poruke

| Poruka                | Namena                     |
|-----------------------|----------------------------|
| TaskMessage           | Master → Worker            |
| ResultMessage         | Worker → Master/Collector  |
| WorkerStatusMessage   | Worker → Master            |
| RegisterWorkerMessage | Worker → Master pri startu |
| ShutdownMessage       | System → Actor pri gašenju |

## Promjena ponašanja (Become)

WorkerActor demonstrira Become mehanizam kroz dvije funkcije ponašanja:

- idleBehavior – početno stanje; prihvata TaskMessage i prelazi u BUSY
- busyBehavior – odbija nove taskove; nakon završetka poziva `ctx.become(idleBehavior)`

## Scheduling

Distribucija taskova se vrši jednostavnim round-robin pristupom između dostupnih workera.

## Lifecycle ponašanje

Sistem podržava:

- inicijalizaciju aktora (onStart)
- gašenje aktora (onStop)
- logovanje lifecycle događaja

## Implementacioni detalji

| Komponenta    | Tehnologija                      |
|---------------|----------------------------------|
| Programski    | Go 1.22+                         |
| RPC transport | gRPC + Protocol Buffers          |
| Testiranje    | Go standardna testing biblioteka |
| Build         | Go modules                       |

## Zaključak

Projektom se implementira generički aktorski framework i njegova demonstracija kroz distribuirani task processing sistem. Sistem demonstrira sve obavezne elemente:

- asinhronu komunikaciju putem mailbox modela
- promenu ponašanja aktora (Become mehanizam)
- lifecycle događaje (onStart / onStop)
- udaljenu komunikaciju između aktora na različitim mašinama